

Cahier des Charges

Enterprise Knowledge Assistant

RAG-based Generative AI Application

Projet :

Application intelligente de recherche documentaire basée sur les LLM

Technologies principales :

Python – FastAPI – Streamlit – LangChain – LangGraph – Mistral API – Docker –
Cloud

Table des matières

1	Présentation du projet	3
1.1	Nom du projet	3
1.2	Contexte	3
2	Objectif général	4
3	Objectifs techniques	5
3.1	Gestion des documents	5
3.2	Recherche intelligente RAG	5
3.3	Génération de réponse avec LLM	6
4	Architecture cible	7
5	Technologies utilisées	9
5.1	Backend	9
5.2	Frontend	9
5.3	Generative AI	9
5.4	Base vectorielle	9
5.5	Data Processing	9
5.6	DevOps	10
5.7	Déploiement	10
6	Architecture logicielle	11
7	Fonctionnalités avancées	12
7.1	Historique des conversations	12
7.2	Authentification	12
7.3	Evaluation du système RAG	12
7.4	Monitoring	12
7.5	CI/CD	13
8	Livrables attendus	14
8.1	Application fonctionnelle	14
8.2	Application Dockerisée	14
8.3	Déploiement public	14
8.4	Documentation GitHub	14

9	Planning	15
9.1	Phase 1 – RAG Core (Jours 1-5)	15
9.2	Phase 2 – Application (Jours 6-10)	15
9.3	Phase 3 – Industrialisation (Jours 11-17)	15
9.4	Phase 4 – Déploiement (Jours 18-21)	15
10	Compétences acquises pour le CV	16

Chapitre 1

Présentation du projet

1.1 Nom du projet

Enterprise Knowledge Assistant – RAG-based Generative AI Application

1.2 Contexte

Les entreprises possèdent aujourd’hui de grandes quantités de documents internes :

- procédures ;
- documentation technique ;
- guides utilisateurs ;
- politiques internes ;
- rapports ;
- contrats.

La recherche manuelle dans ces documents est souvent longue et inefficace.

L’objectif de ce projet est de développer une application basée sur l’Intelligence Artificielle Générative permettant aux utilisateurs d’interroger leurs documents en langage naturel.

Le système devra comprendre les questions des utilisateurs et générer des réponses pertinentes en s’appuyant uniquement sur les informations présentes dans les documents fournis.

Chapitre 2

Objectif général

L'objectif principal est de concevoir et déployer une application GenAI complète basée sur une architecture :

RAG (Retrieval Augmented Generation)

Cette application devra permettre :

- l'ingestion automatique de documents ;
- la recherche sémantique dans une base vectorielle ;
- la génération de réponses avec un LLM ;
- l'exposition d'une API backend ;
- la création d'une interface utilisateur ;
- le déploiement cloud avec Docker.

Chapitre 3

Objectifs techniques

3.1 Gestion des documents

L'utilisateur devra pouvoir :

- charger des documents PDF ;
- extraire automatiquement leur contenu ;
- nettoyer le texte extrait ;
- découper les documents en morceaux appelés *chunks*.

Pipeline d'ingestion :

```
PDF
|
Text Extraction
|
Cleaning
|
Chunking
|
Embedding Generation
|
Vector Database
```

3.2 Recherche intelligente RAG

Lorsqu'un utilisateur pose une question, par exemple :

"Quelle est la procédure de récupération d'un mot de passe ?"

Le système doit :

1. analyser la question ;
2. rechercher les passages pertinents ;
3. récupérer les documents correspondants ;
4. envoyer le contexte au modèle de langage.

3.3 Génération de réponse avec LLM

Le modèle devra :

- répondre uniquement à partir des informations disponibles ;
- limiter les hallucinations ;
- fournir les sources utilisées.

Exemple :

R p o n s e :

Le mot de passe peut être initialisé depuis
le portail interne.

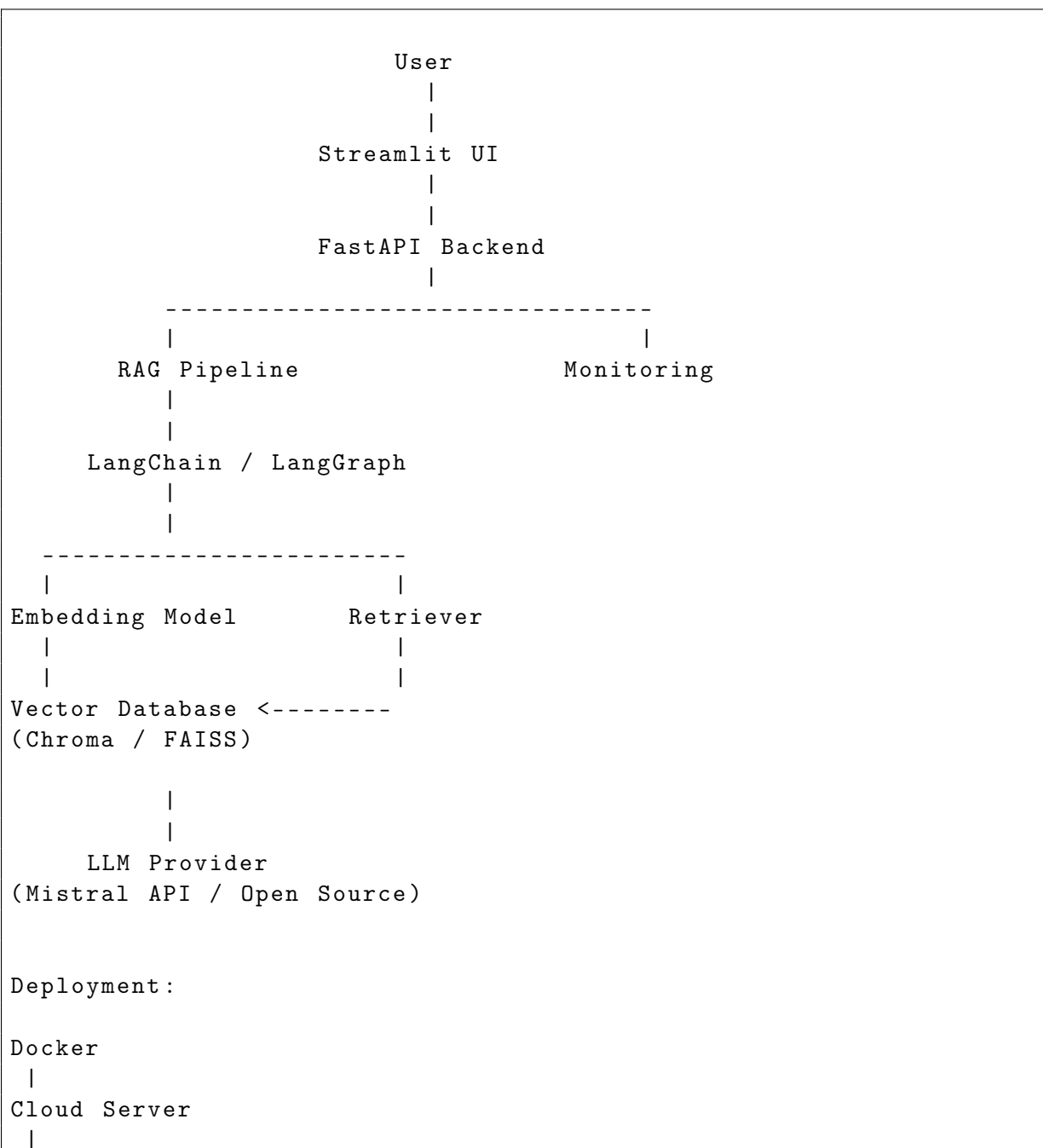
Source :

Security_Policy.pdf

Page 12

Chapitre 4

Architecture cible



AWS / Render

Chapitre 5

Technologies utilisées

5.1 Backend

- Python
- FastAPI
- Pydantic
- Uvicorn

5.2 Frontend

- Streamlit

5.3 Generative AI

- LangChain
- LangGraph
- Mistral API
- Prompt Engineering
- Architecture RAG

5.4 Base vectorielle

Solutions envisagées :

- ChromaDB
- FAISS

5.5 Data Processing

- PyPDF
- LangChain Document Loaders

5.6 DevOps

- Docker
- Docker Compose
- Git
- GitHub

5.7 Déploiement

Deux options :

- Render
- AWS EC2

Chapitre 6

Architecture logicielle

Organisation du projet :

```
enterprise-ai-assistant/  
  
|  
|-- backend/  
|   |-- main.py  
|   |-- api/  
|   |-- services/  
|   |-- rag/  
|   |-- config/  
|  
|-- frontend/  
|   |-- app.py  
|  
|-- data/  
|   |-- documents/  
|  
|-- vectorstore/  
|  
|-- docker/  
|  
|-- tests/  
|  
|-- requirements.txt  
|  
|-- Dockerfile  
|  
|-- docker-compose.yml  
|  
|-- README.md
```

Chapitre 7

Fonctionnalités avancées

7.1 Historique des conversations

Ajout d'un système de sauvegarde des conversations :

- PostgreSQL ;
- Redis.

7.2 Authentification

Gestion des utilisateurs :

- Login ;
- Gestion des droits d'accès.

7.3 Evaluation du système RAG

Mesures :

- pertinence des réponses ;
- temps de réponse ;
- qualité du retrieval.

Outil :

- RAGAS.

7.4 Monitoring

Collecte des informations :

- nombre de questions ;
- erreurs ;
- temps de réponse ;
- consommation de tokens.

Technologies :

- PostgreSQL ;
- Grafana.

7.5 CI/CD

Pipeline proposé :

Git Push

Tests automatiques

Docker Build

Deploy Cloud

Avec :

- GitHub Actions.

Chapitre 8

Livrables attendus

8.1 Application fonctionnelle

L'application devra fournir :

- une interface utilisateur ;
- une API backend ;
- un pipeline RAG complet ;
- une intégration LLM.

8.2 Application Dockerisée

Commande de lancement :

```
docker compose up
```

8.3 Déploiement public

Exemple :

```
https://enterprise-ai-assistant.com
```

8.4 Documentation GitHub

Le README devra contenir :

- description du projet ;
- architecture ;
- installation ;
- utilisation ;
- captures d'écran ;
- choix techniques.

Chapitre 9

Planning

9.1 Phase 1 – RAG Core (Jours 1-5)

Objectif :

Créer un système RAG fonctionnel.

Compétences :

- embeddings ;
- chunking ;
- vector search ;
- prompt engineering.

9.2 Phase 2 – Application (Jours 6-10)

Développement :

- API FastAPI ;
- interface Streamlit.

9.3 Phase 3 – Industrialisation (Jours 11-17)

Ajout :

- Docker ;
- Docker Compose ;
- tests automatisés.

9.4 Phase 4 – Déploiement (Jours 18-21)

Déploiement :

- serveur cloud ;
- HTTPS ;
- monitoring.

Chapitre 10

Compétences acquises pour le CV

Après réalisation du projet :

Generative AI Engineer Project – Enterprise Knowledge Assistant

Compétences démontrées :

- LLM Application Development
- RAG Architecture
- LangChain / LangGraph
- Mistral API
- Vector Databases
- FastAPI Backend
- Docker Deployment
- Cloud Deployment
- AI Application Monitoring